ESILV – Introduction au High Performance Computing

Projet HPC

Traitement parallèle d'un jeu d'images scientifiques

Rôle	Responsabilité	Membre
1. Performance & Vectorization	Baseline, profilage, vectorisation NumPy	Loïs Halmaert
2. Multithreading	Threads : I/O contre calcul, GIL	Arthur Belloc
3. Multiprocessing	Processus, speedup et efficacité	Vladimir Varenne Welnovski
4. MPI	Distribution avec mpi4py, scaling	Kylian Payan

1 Contexte et protocole expérimental

1.1 L'application étudiée

Le programme fourni traite une collection d'images scientifiques selon le pipeline suivant :

$$\text{Chargement} \rightarrow \text{Normalisation} \rightarrow \text{Filtre moyenneur } 3 \times 3 \rightarrow \text{Statistiques} \rightarrow \text{Agrégation globale}$$

Les trois étapes de calcul sont volontairement écrites avec des boucles Python explicites, pixel par pixel. L'objectif du projet est d'accélérer ce programme avec les techniques vues en TP, sans changer les résultats numériques.

1.2 Jeu de données

— 24 images synthétiques de 256×256 pixels (65 536 pixels par image), en `float32`, enregistrées au format `.npy`.
— Générées de façon déterministe (graine `seed=42`) : tout le monde obtient exactement les mêmes images.

```
python sequential_pipeline.py -generate-demo
```

1.3 Machines de mesure

Machine	Caractéristiques
PC portable de Loïs (Acer)	Windows 11, Intel Core 11e génération (Tiger Lake), 4 cœurs physiques / 8 logiques, 2,8 GHz, 16 Go de RAM, Python 3.12, NumPy 2.5
PC portable d'Arthur	Arch Linux, Intel Core i5-6300HQ (2,3 GHz), 2 CPU logiques disponibles (`nproc` = 2), 16 Go de RAM, Python 3.14 (avec GIL), NumPy 2.5
Machine commune pour le tableau final	PC portable d'Arthur (sauf la ligne MPI)

1.4 Référence de correction

La version séquentielle donne toujours le même résumé global. Toute version optimisée doit retrouver ces valeurs :

Grandeur	Valeur de référence
Moyenne des moyennes d'images	0.46763534
Maximum global	1.00000000
Nombre total de pixels au-dessus du seuil (0,65)	145595
Fraction moyenne au-dessus du seuil	0.09256681

2 Parties 1 et 2 – Baseline, profilage et vectorisation (Rôle 1)

2.1 Baseline et profilage

Sur le PC de mesure (Intel Core 11e gén., 4 cœurs / 8 threads, 16 Go), trois exécutions de `sequential_pipeline.py` donnent 2,840 / 2,725 / 2,839 s, soit $T_1 = 2{,}84$ s (médiane, écart $\approx 4\,\%$), avec un résumé global identique à chaque fois. Le profilage `cProfile` (proportions uniquement : le profileur ralentit l'exécution à 5,1 s) donne :

Fonction (24 appels chacune)	Part du temps	Séquentiel, 24 images (s)	Vectorisé, 24 images (s)
`mean_filter_3x3`	$\approx 77\,\%$	2,499	0,0146 ($\times 172$)
`normalize_image`	$\approx 12\,\%$	0,351	0,0027 ($\times 128$)
`compute_image_statistics`	$\approx 9\,\%$	0,208	0,0015 ($\times 143$)
`load_image`	$\approx 0{,}3\,\%$	0,016	inchangé

Temps par étape : médiane de 3 exécutions avec `python vectorized_version.py -compare`, qui vérifie aussi l'égalité des résultats.

Le filtre 3×3 est le goulot principal : il lit 9 valeurs par pixel dans quatre boucles Python imbriquées. Le programme est limité par le calcul, pas par les fichiers. D'après la loi d'Amdahl, n'accélérer que le filtre plafonnerait le gain à $\approx \times 4$: il faut traiter les trois étapes de calcul.

2.2 Vectorisation NumPy

`vectorized_version.py` remplace les boucles par des opérations sur l'image entière, le reste du programme étant réutilisé tel quel :

- normalisation : (image − min)/écart en une seule opération ;
- filtre 3×3 : somme de 9 copies décalées de l'image (découpage par tranches), puis division par 9 ;
- statistiques : `sum()`, `max()` et `count_nonzero(image ≥ seuil)`.

Résultats identiques au bit près : les calculs sont faits en `float64` puis rangés en `float32`, et les 9 voisins sont additionnés dans le même ordre que les boucles (vérifié par `np.array_equal` sur les 24 images ; résumé global identique).

Pipeline complet (3 exécutions) : 0,0395 / 0,0346 / 0,0384 s, soit 0,038 s contre 2,84 s : speedup $\approx \times 74$ sur un seul cœur.

2.3 Analyse

- Toutes les étapes de calcul gagnent un facteur > 100 ; le filtre gagne le plus ($\times 172$) car c'est lui qui faisait le plus de travail dans l'interpréteur. NumPy exécute la boucle en code compilé, sur des données contiguës, avec des instructions SIMD.
- Le goulot change de place : le pipeline complet ne gagne que $\times 74$ contre $\times 163$ pour le calcul seul, car le chargement des fichiers (≈ 16 ms, inchangé) pèse désormais une part importante du temps restant, contre 0,3 % avant.
- Conséquence pour la suite : la vectorisation est complémentaire du parallélisme (fonctions réutilisables en multiprocessing et MPI), mais avec moins de 2 ms de calcul par image, les coûts de lancement et de communication limiteront les gains.

3 Partie 3 – Multithreading (Rôle 2)

Fichier : `threading_version.py`. Responsable : Arthur Belloc.

3.1 Implémentation

Un `ThreadPoolExecutor` applique `process_one_image` (fonctions de référence, ou vectorisées avec `-vectorized`) à chaque fichier ; chaque thread traite des images entières. `map` conserve l'ordre des résultats, et l'agrégation et le CSV réutilisent le code de référence. Le mode `-experiments` isole l'I/O et le calcul avec 1, 2, 4 et 8 threads : chargement réel (fichiers en cache), chargement avec une latence simulée de 50 ms par fichier (disque réseau), calcul seul en boucles et en NumPy (images chargées hors de la zone mesurée), pipeline complet. Chaque temps est la médiane de 3 à 5 runs après une chauffe ; le ratio *temps CPU / temps réel* (`process_time` / `perf_counter`) indique combien de cœurs travaillent réellement en même temps.

3.2 Résultats (PC d'Arthur)

S : speedup par rapport à 1 thread ; C/W : ratio temps CPU / temps réel.

	I/O + latence 50 ms			**Calcul, boucles**			**Pipeline vectorisé**		
Threads	Temps	S	C/W	Temps	S	C/W	Temps	S	C/W
1	1,232 s	1,00	0,03	4,126 s	1,00	1,00	61,6 ms	1,00	0,99
2	0,621 s	1,98	0,05	4,245 s	0,97	0,99	41,6 ms	**1,48**	1,53
4	0,315 s	3,91	0,10	4,272 s	0,97	0,99	51,7 ms	1,19	1,36
8	0,166 s	**7,43**	0,22	4,316 s	0,96	0,99	48,6 ms	1,27	1,57

Chargement réel sans latence (9 ms au total) : $\times 1{,}4$ au mieux. Calcul NumPy seul : $\times 1{,}29$ à 2 threads. Pipeline complet à 2 threads : 4,26 s en boucles (contre 4,06 s en séquentiel), 0,044 s en vectorisé ; les CSV sont identiques à la référence (`diff` vide).

3.3 Analyse

En CPython, un seul thread à la fois peut exécuter du bytecode Python (GIL).

- *I/O avec attente* : le thread relâche le GIL pendant l'attente, donc les attentes se superposent : $\times 7{,}4$ avec 8 threads *sur 2 cœurs*, avec un ratio C/W de 0,03 à 0,22. Ce n'est pas du calcul parallèle. Sans latence, le travail est trop court pour amortir les threads.

— *Calcul en Python pur* : aucun gain. Le ratio C/W reste à 0,99 alors que 2 cœurs sont libres : un seul cœur travaille, les threads se passent le GIL ; la légère perte vient des changements de contexte.
— *Calcul NumPy* : NumPy relâche le GIL dans son code C, d'où un parallélisme partiel (C/W $\approx 1{,}55$, $\times 1{,}3$ à $\times 1{,}5$), limité par le code Python entre les appels NumPy ($\approx 1{,}7$ ms de calcul par image), les 2 cœurs et la bande passante mémoire.

Comme le calcul représente plus de 99 % du temps, le threading seul n'apporte rien ici ($\times 0{,}95$) ; il ne serait utile que pour masquer des lectures lentes.

4 Partie 4 – Multiprocessing (Rôle 3)

Fichier : `multiprocessing_version.py`. Responsable : Vladimir.

4.1 Implémentation

Un `multiprocessing.Pool` (démarrage `forkserver`, défaut de Python 3.14 sous Linux) distribue les *chemins* des images avec `pool.map(..., chunksize=1)` : chaque processus charge et traite ses images, et seuls les dictionnaires de statistiques reviennent au processus principal, qui agrège et écrit le CSV. Le mode `-experiments` vérifie la correction, mesure le coût d'un pool vide, puis T_p pour $p = 1, 2, 4, 8$ (médiane de 3 à 10 runs), avec Speedup$(p) = T_1/T_p$, Efficacité(p) = Speedup$(p)/p$ et la métrique de Karp-Flatt $e = (1/S - 1/p)/(1 - 1/p)$.

4.2 Résultats (PC d'Arthur)

« vs séq. » : gain par rapport à une simple boucle sans pool (4,219 s en boucles, 58 ms en vectorisé).

	Pool vide	**Pipeline, boucles**					**Pipeline, vectorisé**		
p	Coût	T_p	Speedup	Effic.	e	vs séq.	T_p	Speedup	vs séq.
1	25,9 ms	4,233 s	1,00	1,00	–	1,00	91,8 ms	1,00	0,63
2	26,6 ms	2,349 s	**1,80**	0,90	0,11	1,80	77,7 ms	1,18	0,75
4	34,3 ms	2,343 s	1,81	0,45	0,41	1,80	107,4 ms	0,86	0,54
8	82,8 ms	2,397 s	1,77	0,22	0,50	1,76	167,5 ms	0,55	0,35

Résultats identiques à la référence avec 8 processus, en boucles comme en vectorisé.

4.3 Analyse

Chaque processus a son propre interpréteur et son propre GIL : $\times 1{,}80$ avec 2 processus (efficacité 0,90), là où les threads n'apportaient rien. Le gain n'est pas proportionnel au-delà :

— *Nombre de cœurs* : la machine n'a que 2 CPU logiques, donc le speedup plafonne à $\approx 1{,}8$ et l'efficacité chute à 0,45 puis 0,22.
— *Coûts fixes* : un pool coûte 26 à 83 ms (lancement des processus, import de NumPy, sérialisation `pickle`). Karp-Flatt *augmente* avec p (0,11 $\rightarrow$ 0,50) : c'est un surcoût qui croît avec le nombre de processus, pas une partie séquentielle fixe.
— *Granularité* : en vectorisé, tout le calcul tient en 58 ms, du même ordre que le coût du pool, donc le multiprocessing est *plus lent* que la boucle simple. Envoyer les images elles-mêmes plutôt que leurs chemins est encore plus lent (83,7 ms au mieux).
— *Répartition* : avec 8 processus, chacun n'a que 3 images pour amortir le démarrage.

5 Partie 5 – MPI (Rôle 4)

Fichier : `mpi_version.py`. Responsable : Kylian Payan.

5.1 Implémentation

J'ai réparti les 24 images entre les processus MPI. Le rang 0 prépare les groupes et les envoie avec `scatter()`. `bcast()` partage le nombre d'images. Chaque processus traite ses images de son côté. Ensuite, `gather()` rassemble les résultats et `reduce()` récupère le nombre d'images traitées et le temps le plus long. Le rang 0 enregistre le CSV. J'ai gardé les fonctions du programme séquentiel pour obtenir les mêmes résultats.

5.2 Résultats

J'ai lancé le programme avec 1, 2, 4 et 8 processus, en répétant chaque test 3 fois avec la commande `mpiexec -n 4 python mpi_version.py -repeat 3` (ici pour 4 processus).

Strong scaling 24 images au total

Rangs	Temps (s)	Speedup	Efficacité
1	3,766029	1,00	100 %
2	1,783469	2,11	105,6 %
4	0,891277	4,23	105,6 %
8	0,556587	6,77	84,6 %

Scaling faible 24 images par rang

Rangs	Images	Temps (s)	Efficacité
1	24	3,645903	100 %
2	48	3,617989	100,8 %
4	96	3,865161	94,3 %
8	192	5,591005	65,2 %

5.3 Analyse

Strong scaling : avec 24 images, le temps passe de 3,77 s à 0,56 s entre 1 et 8 processus.
Le speedup atteint 6,77 et l'efficacité 84,6 %. Le gain reste important, même s'il n'est
pas parfaitement proportionnel au nombre de processus.
Scaling faible : avec 24 images par processus, le temps reste proche de 3,6 s jusqu'à 4 processus.
8 processus, il atteint 5,59 s et l'efficacité baisse à 65,2 %. Le partage des ressources
et les surcoûts MPI peuvent expliquer cette hausse.

6 Comparaison des versions

Machine utilisée : PC portable d'Arthur (2 CPU logiques), sauf la ligne MPI, mesurée par Kylian sur sa machine (speedup par rapport à son propre T_1).

Version	Temps d'exécution (s)	Speedup	Résumé identique ?
Séquentiel	4,06	1,00	oui (référence)
Vectorisé	0,058	70	oui
Threading (2 threads)	4,26	0,95	oui
Multiprocessing (2 processus)	2,35	1,73	oui
MPI (8 rangs, machine de Kylian)	0,56	6,77	oui
Combinaison vectorisé + threading (2 threads)	0,042	98	oui
Combinaison vectorisé + multiprocessing (2 processus)	0,078	52	oui

Sur le PC portable de Loïs, à titre indicatif : séquentiel 2,84 s ; vectorisé 0,038 s, soit un speedup d'environ $\times 74$.

7 Conclusion

Quelle stratégie d'optimisation ou de parallélisation est la plus adaptée à cette application, et pourquoi ?

Éléments établis par les parties 1 et 2 :

- L'application est limitée par le calcul (99 % du temps), pas par les fichiers.
- La vectorisation seule divise déjà le temps de calcul par plus de 100 sur un seul cœur.
- Le parallélisme (processus, MPI) peut s'ajouter à la vectorisation, mais son gain sera limité par le faible temps restant par image et par le coût de lancement et de communication.

Les parties 3 à 5 le confirment. La **vectorisation** est la stratégie la plus adaptée : elle apporte à elle seule un facteur 70 à 74 sur un seul cœur, car le problème venait de l'interpréteur Python et non du manque de cœurs ; aucune version parallèle à boucles n'en approche (au mieux $\times 5{,}92$ avec 8 rangs MPI). Sur nos 24 petites images, ajouter du parallélisme au code vectorisé rapporte peu : un petit bonus avec les threads (NumPy relâche le GIL et les threads coûtent peu à créer), une perte avec les processus, dont le coût de démarrage dépasse le calcul. La meilleure combinaison mesurée est donc **vectorisation + threads** (0,042 s, $\times 98$). Pour un jeu de données beaucoup plus grand, nous recommandons **vectorisation + multiprocessing** sur une machine, ou **vectorisation + MPI** sur plusieurs nœuds, avec de gros lots d'images par worker pour amortir les coûts fixes ; les threads restent utiles pour masquer des I/O lentes.

Annexe – Contributions individuelles

Rôle 1 – Loïs Halmaert (Performance & Vectorization)

- Implémentation : `vectorized_version.py` (normalisation, filtre 3×3 et statistiques vectorisés, mode `-compare` de mesure par étape avec vérification de correction).
- Expériences : mesure de T_1 (3 exécutions), profilage `cProfile`, comparaison séquentiel / vectorisé fonction par fonction.
- Résultats : $T_1 = 2{,}84$ s ; le filtre 3×3 représente 77 % du temps ; la vectorisation accélère les trois étapes de calcul d'un facteur 128 à 172, avec des résultats identiques au bit près ; le pipeline complet passe de 2,84 s à 0,038 s ($\times 74$).

Rôle 2 – Arthur Belloc (Multithreading)

- Implémentation : `threading_version.py` (pipeline avec `ThreadPoolExecutor`, option `-vectorized`, mode `-experiments`) et `utils/benchmark.py` (médiane, CV).
- Expériences : I/O réelle et avec latence, calcul en boucles et NumPy, pipeline complet, avec 1 à 8 threads et ratio CPU/temps réel ; exécution des mesures multiprocessing sur ma machine.
- Résultats : $\times 7{,}4$ sur l'I/O avec latence, aucun gain sur le Python pur (GIL), $\times 1{,}3$ à $\times 1{,}5$ en NumPy ; résultats identiques à la référence.
- Ce que j'ai appris : les threads servent à attendre en parallèle, pas à calculer en parallèle ; mesurer proprement (chauffe, médiane, séparer I/O et calcul) et utiliser le ratio CPU/temps réel pour voir combien de cœurs travaillent vraiment.

Rôle 3 – Vladimir (Multiprocessing)

- Implémentation : `multiprocessing_version.py` (`Pool.map` sur les chemins, méthode de démarrage configurable, mode `-experiments` : correction, coût du pool, speedup, efficacité, Karp-Flatt).
- Résultats : $\times 1{,}80$ avec 2 processus en boucles ; en vectorisé, le coût du pool dépasse le calcul.

Rôle 4 – Kylian Payan (MPI)

- Implémentation : j'ai réparti les images avec mpi4py et rassemblé les résultats.
- Expériences : strong scaling sur 24 images, puis scaling faible avec 24 images par processus (24, 48, 96 et 192 images), avec 3 répétitions par test.
- Résultats : en strong scaling, 3,77 s à 0,56 s (speedup 6,77) ; en scaling faible, 3,65 s à 5,59 s, soit 65,2 % d'efficacité à 8 processus.
- Ce que j'ai appris : répartir le travail avec MPI et analyser le passage à l'échelle.